

PyRE Framework

Arquitectura Modular para Rule Extraction

Documento Técnico Completo

Versión: 1.0

Fecha: Noviembre 2025

Autor: Jaume - Quantdemy

Framework: Python Rule Extraction (PyRE)

PARTE I: Resumen Ejecutivo

Concepto Central

Problema: MVP actual es un monolito - difícil de personalizar y extender.

Solución: Arquitectura modular plug-and-play donde cada alumno puede elegir opciones por módulo según sus necesidades.

Los 9 Módulos del Framework

DATA → TARGET → INDICATORS → FEATURE SELECTION → EXTRACTION



VALIDATION → ENSEMBLE → FILTERS → PARSER →  Deploy

#	Módulo	Opciones Disponibles	Estado
1	DATA	CSV / Quandl API / MT5 API	✓ CSV listo
2	TARGET	Retorno Simple / IBS / Ajustado por Costes	✓ 2/3 listos
3	INDICATORS	CSV / MT5 Clone / pandas_ta	✓ Listo
4	FEATURE SELECTION	OneR / Mutual Info / Correlation / Lasso	✓ 3/4 listos
5	EXTRACTION	OneR Modificado / Decision Trees / Association Rules	✓ OneR listo
6	VALIDATION	Monkey Test / Synthetic Data / Cross-Val	✓ 2/3 listos
7	ENSEMBLE	OR / AND / Voting / Weighted	✓ OR listo
8	FILTERS	ADX / ATR / Time Filters	⌚ Pendiente
9	PARSER	MQL5 / Python Bot / TradingView	⌚ Pendiente



Cómo Funciona

Configuración = Pipeline

Los alumnos definen su pipeline con un archivo YAML:

```
data:
  source: csv          # ← Elige CSV, Quandl o MT5
  file_path: mi_data.csv

target:
  method: change_target  # ← Elige change_target o ibs_target
  days_ahead: 5

feature_selection:
  method: oner          # ← Elige oner, mutual_info, correlation
  top_n: 10

# ... más configuración
```

Ejecución = 1 Línea

```
from pyre import PyREPipeline

pipeline = PyREPipeline('mi_config.yaml')
results = pipeline.run()
```



Beneficios para Alumnos

Perfil	Beneficio
Principiante	Configs predefinidas - solo elegir opciones
Intermedio	Combinar módulos - experimentar sin código
Avanzado	Crear módulos custom - contribuir al framework



3 Flujos Típicos

Flujo A: CSV → MT5 EA

CSV Upload → Retorno Simple → CSV Indicators → OneR Selection
→ OneR Extraction → Full Validation → OR Ensemble → ADX Filter → MQL5

Flujo B: Research Rápida

Quandl API → IBS Target → pandas_ta → Mutual Info
→ OneR Extraction → Monkey Test → Weighted Ensemble → Python Bot

Flujo C: MT5 Live Trading

MT5 API → Retorno Simple → MT5 Clone → OneR Selection
→ OneR Extraction → Full Validation → OR Ensemble → ADX+ATR → MQL5 EA



Estructura del Código

```
pyre/
├─ data/           # Módulo 1: Ingesta de datos
├─ target/         # Módulo 2: Cálculo de target
├─ indicators/     # Módulo 3: Indicadores técnicos
├─ feature_selection/ # Módulo 4: Selección de features
├─ extraction/     # Módulo 5: Extracción de reglas
├─ validation/     # Módulo 6: Validación estadística
├─ ensemble/       # Módulo 7: Combinación de reglas
├─ filters/        # Módulo 8: Filtros de mercado
├─ parser/         # Módulo 9: Generación de código
└─ pipeline.py     # Orquestador principal
```



Roadmap

Fase	Timeline	Objetivo
Fase 1	Semanas 1-2	Estructura y preparación
Fase 2	Semanas 3-6	Migrar módulos core
Fase 3	Semanas 7-8	Factories y pipeline
Fase 4	Semanas 9-10	UI Streamlit nueva
Fase 5	Semanas 11-12	Testing y release v1.0



Principios de Diseño

1. **Interface-First:** Cada módulo define interface clara
2. **Plug & Play:** Opciones intercambiables sin cambiar código
3. **Config-Driven:** Pipeline definido por YAML, no código
4. **Extensible:** Fácil añadir nuevas implementaciones
5. **Testeable:** Cada módulo testeable independientemente



Para Alumnos: Cómo Empezar

1. Usa Configs Predefinidas

```
# Opción fácil - usar template
pyre run --config templates/csv_to_mql5.yaml
```

2. Personaliza Tu Pipeline

```
# Copia template y modifica
cp templates/csv_to_mql5.yaml mi_config.yaml
# Edita mi_config.yaml con tus preferencias
pyre run --config mi_config.yaml
```

3. Crea Tu Propio Módulo (avanzado)

```
# pyre/feature_selection/mi_selector.py
from pyre.feature_selection.base import FeatureSelector

class MyCustomSelector(FeatureSelector):
    def select_top_features(self, df, target_col, top_n):
        # Tu implementación aquí
        pass

# Registrar en Factory
# Ya está listo para usar en configs!
```

Decisiones Importantes

¿Qué módulos usar?

Escenario	Recomendación
Quiero deploy en MT5	DATA=CSV, INDICATORS=MT5Clone, PARSER=MQL5
Solo investigación	DATA=Quandl, INDICATORS=pandas_ta, PARSER=Python
Máxima velocidad	FEATURE_SELECTION=Correlation, VALIDATION=MonkeyTest
Máxima robustez	VALIDATION=All, ENSEMBLE=Weighted

? FAQ Rápido

Q: ¿Tengo que migrar ya?

A: No. MVP actual sigue funcionando. Migración opcional.

Q: ¿Puedo usar solo algunos módulos?

A: Sí. Cada módulo es independiente.

Q: ¿Cómo añado mis propios indicadores?

A: Añádelos como columnas en el CSV o crea módulo custom.

Q: ¿Funciona con múltiples activos?

A: Actualmente un activo a la vez. Multi-asset es roadmap futuro.



Comparación MVP vs Modular

Aspecto	MVP Actual	Arquitectura Modular
Archivos	2 (utils.py, app.py)	50+ organizados
Líneas de código	6,000+ monolítico	6,000+ modularizado
Personalización	Editar código	Cambiar config
Añadir features	Modificar utils.py	Crear nuevo módulo
Testing	Manual	Automatizado
Colaboración	Difícil (conflictos)	Fácil (módulos aislados)
Curva aprendizaje	Empinada	Gradual
Mantenibilidad	Baja	Alta

PARTE II: Libro Blanco Técnico

Resumen Ejecutivo

PyRE es un framework modular diseñado para la extracción, validación y despliegue de reglas de trading algorítmico. Su arquitectura basada en módulos conectables permite a cada usuario personalizar su pipeline según sus necesidades específicas, manteniendo la rigurosidad metodológica del proceso de Rule Extraction.

Características principales:

-  Arquitectura modular y extensible
 -  Múltiples opciones por módulo (plug-and-play)
 -  Validación estadística rigurosa
 -  Compatible con múltiples fuentes de datos
 -  Exportación a MT5 (MQL5)
-

Tabla de Contenidos

1. [Arquitectura General](#)
2. [Módulo 1: DATA](#)
3. [Módulo 2: TARGET](#)
4. [Módulo 3: INDICATORS](#)
5. [Módulo 4: FEATURE SELECTION](#)
6. [Módulo 5: EXTRACTION](#)
7. [Módulo 6: VALIDATION](#)
8. [Módulo 7: ENSEMBLE](#)
9. [Módulo 8: FILTERS](#)
10. [Módulo 9: PARSER](#)
11. [Flujos de Trabajo Típicos](#)
12. [Guía de Implementación](#)
13. [Roadmap y Extensiones Futuras](#)

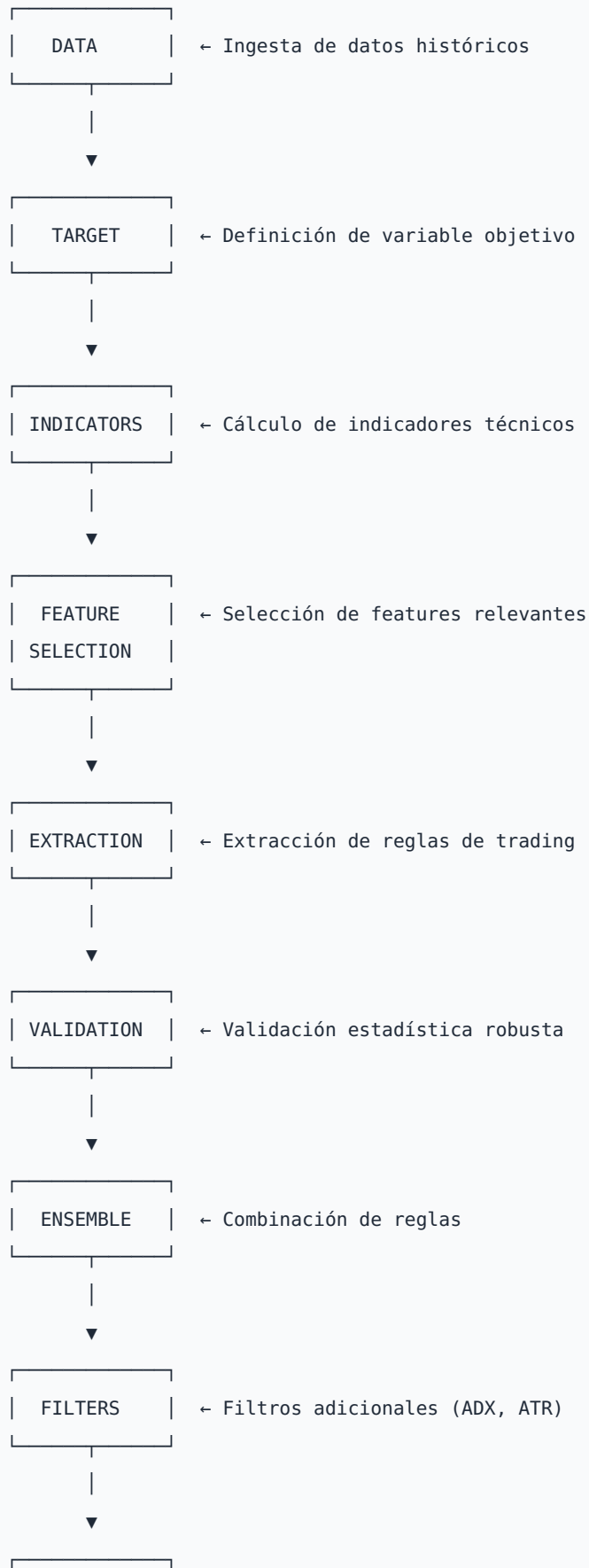
1. Arquitectura General

1.1. Filosofía del Diseño

El framework PyRE está diseñado bajo los siguientes principios:

1. **Modularidad:** Cada fase del pipeline es un módulo independiente
2. **Intercambiabilidad:** Múltiples implementaciones por módulo
3. **Composabilidad:** Los módulos se conectan mediante interfaces estándar
4. **Extensibilidad:** Fácil añadir nuevas opciones sin romper el código existente

1.2. Pipeline General



PARSER

← Exportación a MQL5 / Python

1.3. Estructura de Datos Estándar

Cada módulo trabaja con un DataFrame de pandas con la siguiente estructura mínima:

```
DataFrame:
  Index: Date (datetime)
  Columns:
    - Open, High, Low, Close (OHLC prices)
    - [Indicadores técnicos dinámicos]
    - Target (añadido en módulo TARGET)
```

Contrato entre módulos:

- **Input:** DataFrame con estructura estándar
- **Output:** DataFrame con columnas adicionales o modificadas
- **Metadata:** Diccionario con información del procesamiento

2. Módulo 1: DATA

2.1. Objetivo

Ingerir datos históricos de precios e indicadores desde diferentes fuentes.

2.2. Opciones Disponibles

Opción A: CSV Upload (Drag & Drop)

Ventajas:

- ☒ Sin dependencias externas
- ☒ Control total sobre los datos
- ☒ Útil para datos preprocesados

Requisitos:

- Formato: CSV con columnas `Date,Open,High,Low,Close,[Indicadores]`
- Encoding: UTF-8
- Separador: coma

Implementación:

```
def load_from_csv(file_path):
    """Carga datos desde CSV"""
    df = pd.read_csv(file_path)
    df = clean_data(df) # Limpieza estándar
    return df
```

Opción B: Quandl API**Ventajas:**

- ☒ Datos de calidad profesional
- ☒ Múltiples activos disponibles
- ☒ Actualización automática

Requisitos:

- API Key de Quandl
- Suscripción activa
- Conexión a internet

Implementación:

```
def load_from_quandl(symbol, start_date, end_date, api_key):
    """Carga datos desde Quandl"""
    import quandl
    quandl.ApiConfig.api_key = api_key
    df = quandl.get(symbol, start_date=start_date, end_date=end_date)
    return df
```

Opción C: MetaTrader 5 API**Ventajas:**

- ☒ Datos directos del broker
- ☒ Sincronización con cuenta real
- ☒ Acceso a todos los símbolos del broker

Requisitos:

- MT5 instalado y autenticado
- Credenciales de cuenta
- Biblioteca `MetaTrader5`

Implementación:

```
def load_from_mt5(symbol, timeframe, start_date, end_date, login, password, server):
    """Carga datos desde MT5"""
    import MetaTrader5 as mt5

    mt5.initialize()
    mt5.login(login, password=password, server=server)

    rates = mt5.copy_rates_range(symbol, timeframe, start_date, end_date)
    df = pd.DataFrame(rates)
    df['time'] = pd.to_datetime(df['time'], unit='s')
    df.rename(columns={'time': 'Date', 'open': 'Open',
                      'high': 'High', 'low': 'Low', 'close': 'Close'},
              inplace=True)

    mt5.shutdown()
    return df
```

2.3. Interface Estándar

```
class DataLoader:
    def load(self, **kwargs) -> pd.DataFrame:
        """
        Método abstracto para cargar datos

        Returns:
            pd.DataFrame con estructura OHLC estándar
        """
        pass
```

3. Módulo 2: TARGET

3.1. Objetivo

Definir la variable objetivo que las reglas intentarán predecir.

3.2. Opciones Disponibles

Opción A: Retorno Simple a N días

Fórmula:

$$\text{Target}[t] = (\text{Open}[t+N+1] - \text{Open}[t+1]) / \text{Open}[t+1]$$

Ventajas:

- ☒ Simple y interpretable
- ☒ Captura movimientos direccionales
- ☒ Fácil de validar

Parámetros:

- `days_ahead` : int (default=1) - días de prospección

Implementación:

```
def change_target(df, days_ahead=1):
    """Calcula retorno simple a N días"""
    open_tomorrow = df['Open'].shift(-1)
    open_target_day = df['Open'].shift(-(days_ahead + 1))
    df['Target'] = (open_target_day - open_tomorrow) / open_tomorrow
    return df
```

Opción B: IBS Target a N días

Fórmula:

$$\text{IBS_Target}[t] = (\text{Close_final} - \text{Low_min_período}) / (\text{High_max_período} - \text{Low_min_período})$$

Ventajas:

- ☒ Captura posición relativa en el rango
- ☒ Normalizado entre 0 y 1
- ☒ Menos sensible a outliers

Parámetros:

- `days_ahead` : int (default=1) - días del período

Implementación:

```
def ibs_target(df, days_ahead=1):
    """Calcula IBS extendido a N días"""
    ibs_target = []

    for i in range(len(df)):
        if i + days_ahead >= len(df):
            ibs_target.append(np.nan)
            continue

        start_idx = i + 1
        end_idx = i + days_ahead

        period_high = df['High'].iloc[start_idx:end_idx + 1]
        period_low = df['Low'].iloc[start_idx:end_idx + 1]
        final_close = df['Close'].iloc[end_idx]

        max_high = period_high.max()
        min_low = period_low.min()

        if max_high == min_low:
            ibs_value = 0.5
        else:
            ibs_value = (final_close - min_low) / (max_high - min_low)

        ibs_target.append(ibs_value)

    df['Target'] = ibs_target
    return df
```

Opción C: Retorno Ajustado por Costes (PENDIENTE)**Objetivo:**

Considerar spread, slippage y comisiones como handicap sobre el retorno.

Fórmula propuesta:

$$\text{Target_ajustado} = \text{Retorno} - (\text{Spread} + \text{Slippage} + \text{Comisión})$$
Ventajas:

- ☒ Más realista
- ☒ Filtra reglas que capturan movimientos pequeños
- ☒ Mejora la viabilidad en live trading

Parámetros:

- `spread_pips` : float - spread típico en pips
- `slippage_pips` : float - slippage esperado
- `commission_pct` : float - comisión en %

3.3. Interface Estándar

```
class TargetCalculator:
    def calculate(self, df: pd.DataFrame, **kwargs) -> pd.DataFrame:
        """
        Calcula la variable Target

        Args:
            df: DataFrame con OHLC
            **kwargs: parámetros específicos de cada método

        Returns:
            DataFrame con columna 'Target' añadida
        """
        pass
```

4. Módulo 3: INDICATORS

4.1. Objetivo

Calcular indicadores técnicos que servirán como features para la extracción de reglas.

4.2. Opciones Disponibles

Opción A: Indicadores desde CSV

Ventajas:

- ☒ Sin cálculos adicionales
- ☒ Control total sobre los indicadores
- ☒ Reproducibilidad exacta

Requisitos:

- CSV debe incluir columnas de indicadores
- Nombres de columnas estandarizados

Implementación:

```
def use_csv_indicators(df):  
    """Usa indicadores ya presentes en el CSV"""  
    expected_cols = ['Open', 'High', 'Low', 'Close']  
    indicator_cols = [col for col in df.columns if col not in expected_cols]  
  
    if len(indicator_cols) == 0:  
        raise ValueError("No se encontraron indicadores en el CSV")  
  
    return df
```

Opción B: Clonar Indicadores de MT5

Ventajas:

- ☒ Compatibilidad exacta con EA de MT5
- ☒ Mismo código en backtest y live

Requisitos:

- Conocimiento de la implementación de MT5
- Replicar exactamente el cálculo

Implementación:

```
def calculate_mt5_indicators(df):
    """
    Calcula indicadores idénticos a MT5
    IMPORTANTE: Verificar que el cálculo sea exacto
    """

    def rsi_mt5(series, period=14):
        delta = series.diff()
        gain = (delta.where(delta > 0, 0)).rolling(window=period).mean()
        loss = (-delta.where(delta < 0, 0)).rolling(window=period).mean()
        rs = gain / loss
        rsi = 100 - (100 / (1 + rs))
        return rsi

    df['RSI_14'] = rsi_mt5(df['Close'], 14)
    return df
```

Opción C: Librerías Python (pandas_ta, ta-lib)

Ventajas:

-  Rápido de implementar
-  Amplia variedad de indicadores
-  Bien testeadas

Limitaciones:

-  Pueden diferir de MT5
-  Solo válido si se ejecuta desde Python

Implementación:

```
def calculate_pandas_ta_indicators(df):
    """Calcula indicadores usando pandas_ta"""
    import pandas_ta as ta

    df.ta.rsi(length=14, append=True)
    df.ta.macd(append=True)
    df.ta.bbands(append=True)
    df.ta.atr(length=14, append=True)

    return df
```

4.3. Compatibilidad MT5 vs Python

Decisión crítica:

Escenario	Recomendación
Ejecución vía EA de MT5	Usar Opción B (clonar MT5)
Ejecución desde Python	Opción C (pandas_ta) válida
Máxima flexibilidad	Opción A (CSV) + indicadores pre-calculados

4.4. Interface Estándar

```
class IndicatorCalculator:
    def calculate(self, df: pd.DataFrame, indicators_config: dict) -> pd.DataFrame:
        """
        Calcula indicadores técnicos

        Args:
            df: DataFrame con OHLC
            indicators_config: configuración de indicadores

        Returns:
            DataFrame con indicadores añadidos
        """
        pass
```

5. Módulo 4: FEATURE SELECTION

5.1. Objetivo

Reducir dimensionalidad y seleccionar las features más relevantes para predecir el target.

5.2. Proceso en Dos Fases

Fase 1: Reducción de Multicolinealidad

Objetivo: Eliminar features redundantes altamente correlacionadas.

Métodos disponibles:

- Correlación de Pearson
- Correlación de Spearman
- VIF (Variance Inflation Factor)

Implementación:

```
def reduce_multicollinearity(df, threshold=0.85, method='pearson'):
    """
    Elimina features con correlación > threshold

    Args:
        df: DataFrame con features
        threshold: umbral de correlación
        method: 'pearson', 'spearman', 'kendall'

    Returns:
        DataFrame con features no correlacionadas
    """
    if method == 'pearson':
        corr_matrix = df.corr(method='pearson')
    elif method == 'spearman':
        corr_matrix = df.corr(method='spearman')

    to_drop = set()
    for i in range(len(corr_matrix.columns)):
        for j in range(i+1, len(corr_matrix.columns)):
            if abs(corr_matrix.iloc[i, j]) > threshold:
                to_drop.add(corr_matrix.columns[j])

    return df.drop(columns=to_drop)
```

Fase 2: Selección de Features

Opción A: OneR (Recomendado)

Ventajas:

-  Evalúa capacidad predictiva directa
-  Simple y robusto
-  Alineado con la extracción de reglas

Implementación:

```
def feature_selection_oner(df, target_col='Target', top_n=10):  
    """  
    Selecciona top N features según OneR score  
    """  
    df['Target_Bin'] = pd.qcut(df[target_col], q=5, labels=False, duplicates='drop')  
  
    scores = {}  
    for col in df.columns:  
        if col in ['Target', 'Target_Bin']:  
            continue  
  
        try:  
            df['Feature_Bin'] = pd.qcut(df[col], q=5, labels=False, duplicates='drop')  
        except:  
            continue  
  
        accuracy = (df['Target_Bin'] == df['Feature_Bin']).mean()  
        scores[col] = accuracy  
  
    top_features = sorted(scores.items(), key=lambda x: x[1], reverse=True)[:top_n]  
    return [f[0] for f in top_features]
```

Opción B: Mutual Information

Ventajas:

-  Captura relaciones no lineales
-  Agnóstico a la distribución

Implementación:

```
def feature_selection_mi(df, target_col='Target', top_n=10):  
    """Selecciona features por Mutual Information"""  
    from sklearn.feature_selection import mutual_info_regression  
  
    X = df.drop(columns=[target_col])  
    y = df[target_col]  
  
    mi_scores = mutual_info_regression(X, y)  
    mi_scores = pd.Series(mi_scores, index=X.columns)  
  
    top_features = mi_scores.nlargest(top_n).index.tolist()  
    return top_features
```

Opción C: Correlation-based

Ventajas:

-  Muy rápido
-  Interpretable

Implementación:

```
def feature_selection_correlation(df, target_col='Target', top_n=10):  
    """Selecciona features por correlación absoluta con target"""  
    correlations = df.corrwith(df[target_col]).abs()  
    correlations = correlations.drop(target_col)  
  
    top_features = correlations.nlargest(top_n).index.tolist()  
    return top_features
```


5.3. Interface Estándar

```
class FeatureSelector:
    def select(self, df: pd.DataFrame, target_col: str,
              correlation_threshold: float = 0.85,
              top_n: int = 10) -> List[str]:
        """
        Selecciona features relevantes

        Args:
            df: DataFrame con features y target
            target_col: columna target
            correlation_threshold: umbral para multicolinealidad
            top_n: número de features a seleccionar

        Returns:
            Lista de nombres de features seleccionadas
        """
        df_reduced = self.reduce_multicollinearity(df, correlation_threshold)
        selected_features = self.select_top_features(df_reduced, target_col, top_n)

        return selected_features
```

6. Módulo 5: EXTRACTION

6.1. Objetivo

Generar reglas de trading a partir de las features seleccionadas.

6.2. Método: OneR Modificado con Cuantiles

Algoritmo:

1. Para cada feature seleccionada:
2. Discretizar en bins usando cuantiles
3. Calcular el retorno promedio por bin

4. Identificar bins con retorno significativo

5. Generar reglas del tipo:

```
IF Feature IN [Q_low, Q_high] THEN Signal
```

Implementación:

```

def extract_rules_oner(df, features, target_col='Target', n_quantiles=5,
                      min_samples=30, threshold_percentile=60):
    """
    Extrae reglas usando OneR modificado

    Args:
        df: DataFrame con features y target
        features: lista de features a usar
        target_col: columna target
        n_quantiles: número de bins para discretización
        min_samples: mínimo de muestras por bin
        threshold_percentile: percentil para considerar regla válida

    Returns:
        list de dictionaries con reglas extraídas
    """
    rules = []

    for feature in features:
        try:
            df[f'{feature}_bin'], bins = pd.qcut(df[feature], q=n_quantiles,
                                                labels=False, retbins=True,
                                                duplicates='drop')

        except:
            continue

        bin_stats = df.groupby(f'{feature}_bin')[target_col].agg(['mean', 'count'])
        bin_stats = bin_stats[bin_stats['count'] >= min_samples]

        threshold = bin_stats['mean'].quantile(threshold_percentile / 100)
        good_bins = bin_stats[bin_stats['mean'] >= threshold].index.tolist()

        for bin_idx in good_bins:
            rule = {
                'feature': feature,
                'bin': int(bin_idx),
                'lower_bound': bins[bin_idx],
                'upper_bound': bins[bin_idx + 1],
                'mean_return': bin_stats.loc[bin_idx, 'mean'],
                'n_samples': int(bin_stats.loc[bin_idx, 'count']),
                'rule_str': f"{feature} IN [{bins[bin_idx]:.4f}, {bins[bin_idx+1]:.4f}]"
            }

```

```

rules.append(rule)

return rules

```

6.3. Características de las Reglas

Cada regla contiene:

- `feature` : nombre del indicador
- `bin` : índice del cuantil
- `lower_bound` : límite inferior del rango
- `upper_bound` : límite superior del rango
- `mean_return` : retorno promedio esperado
- `n_samples` : número de ocurrencias históricas
- `rule_str` : representación legible de la regla

6.4. Interface Estándar

```

class RuleExtractor:
    def extract(self, df: pd.DataFrame, features: List[str],
               target_col: str, **kwargs) -> List[Dict]:
        """
        Extrae reglas de trading

        Args:
            df: DataFrame con features y target
            features: lista de features a usar
            target_col: columna target
            **kwargs: parámetros del método

        Returns:
            Lista de diccionarios con reglas
        """
        pass

```

7. Módulo 6: VALIDATION

7.1. Objetivo

Validar robustez de las reglas mediante técnicas estadísticas avanzadas.

7.2. Validación en Dos Capas

Capa 1: Monkey Test (Permutation Test)

Objetivo: Verificar que las reglas no son producto del azar.

Método:

1. Permutar aleatoriamente los targets
2. Aplicar las reglas en datos permutados
3. Comparar performance real vs permutaciones

Implementación:

```

def monkey_test(df, rules, n_permutations=1000, confidence_level=0.95):
    """
    Test de permutaciones para validar reglas

    Args:
        df: DataFrame con features y target
        rules: lista de reglas a validar
        n_permutations: número de permutaciones
        confidence_level: nivel de confianza (0.95 = 95%)

    Returns:
        dict con resultados del test para cada regla
    """
    results = {}

    for rule in rules:
        mask = (df[rule['feature']] >= rule['lower_bound']) & \
            (df[rule['feature']] <= rule['upper_bound'])
        real_return = df.loc[mask, 'Target'].mean()

        permuted_returns = []
        for _ in range(n_permutations):
            df_perm = df.copy()
            df_perm['Target'] = np.random.permutation(df_perm['Target'].values)

            perm_return = df_perm.loc[mask, 'Target'].mean()
            permuted_returns.append(perm_return)

        percentile = (np.sum(np.array(permuted_returns) <= real_return) /
                      n_permutations)

        p_value = 2 * min(percentile, 1 - percentile)
        is_significant = p_value < (1 - confidence_level)

        results[rule['rule_str']] = {
            'real_return': real_return,
            'p_value': p_value,
            'is_significant': is_significant,
            'confidence_level': confidence_level
        }

    return results

```

Capa 2: Data Sintética

Objetivo: Verificar que las reglas funcionan en datos con propiedades similares pero no idénticos.

Método:

1. Generar datos sintéticos conservando propiedades estadísticas
2. Aplicar las reglas en datos sintéticos
3. Comparar performance real vs sintética

Implementación:

```

def synthetic_data_test(df, rules, n_synthetic=1000, method='bootstrap'):
    """
    Validación con datos sintéticos

    Args:
        df: DataFrame original
        rules: lista de reglas
        n_synthetic: número de datasets sintéticos
        method: 'bootstrap' o 'parametric'

    Returns:
        dict con resultados del test
    """
    results = {}

    for rule in rules:
        mask = (df[rule['feature']] >= rule['lower_bound']) & \
            (df[rule['feature']] <= rule['upper_bound'])
        real_return = df.loc[mask, 'Target'].mean()

        synthetic_returns = []

        for _ in range(n_synthetic):
            if method == 'bootstrap':
                df_synth = df.sample(n=len(df), replace=True)
            elif method == 'parametric':
                from scipy import stats
                params = stats.norm.fit(df[rule['feature']])
                synthetic_values = stats.norm.rvs(*params, size=len(df))

                df_synth = df.copy()
                df_synth[rule['feature']] = synthetic_values

            mask_synth = (df_synth[rule['feature']] >= rule['lower_bound']) & \
                (df_synth[rule['feature']] <= rule['upper_bound'])

            if mask_synth.sum() > 0:
                synth_return = df_synth.loc[mask_synth, 'Target'].mean()
                synthetic_returns.append(synth_return)

        ci_lower = np.percentile(synthetic_returns, 2.5)
        ci_upper = np.percentile(synthetic_returns, 97.5)

```



```
is_stable = ci_lower <= real_return <= ci_upper

results[rule['rule_str']] = {
    'real_return': real_return,
    'synthetic_mean': np.mean(synthetic_returns),
    'synthetic_std': np.std(synthetic_returns),
    'ci_lower': ci_lower,
    'ci_upper': ci_upper,
    'is_stable': is_stable
}

return results
```

7.3. Criterios de Aceptación

Una regla es válida si:

1. ☒ Pasa el Monkey Test (p-value < 0.05)
2. ☒ Performance real dentro del IC en datos sintéticos
3. ☒ Mínimo de trades (ej. >= 30)

7.4. Interface Estándar

```
class RuleValidator:
    def validate(self, df: pd.DataFrame, rules: List[Dict],
                 validation_config: dict) -> List[Dict]:
        """
        Valida reglas usando múltiples tests

        Args:
            df: DataFrame con features y target
            rules: lista de reglas a validar
            validation_config: configuración de tests

        Returns:
            Lista de reglas con resultados de validación
        """
        monkey_results = self.monkey_test(df, rules, **validation_config['monkey'])
        synthetic_results = self.synthetic_data_test(df, rules,
                                                    **validation_config['synthetic'])

        validated_rules = []
        for rule in rules:
            rule_str = rule['rule_str']
            rule['validation'] = {
                'monkey_test': monkey_results[rule_str],
                'synthetic_test': synthetic_results[rule_str],
                'is_valid': (monkey_results[rule_str]['is_significant'] and
                           synthetic_results[rule_str]['is_stable'])
            }

            if rule['validation']['is_valid']:
                validated_rules.append(rule)

        return validated_rules
```

8. Módulo 7: ENSEMBLE

8.1. Objetivo

Combinar múltiples reglas validadas para reducir el drawdown y mejorar la estabilidad.

8.2. Método: Lógica OR con Optimización de Drawdown

Algoritmo:

1. Generar todas las combinaciones posibles de reglas
2. Para cada combinación:
3. Aplicar lógica OR (señal si cualquier regla se activa)
4. Simular equity curve
5. Calcular drawdown máximo
6. Seleccionar combinaciones con menor drawdown

Implementación:

```

def ensemble_by_drawdown(df, rules, max_rules_per_ensemble=5):
    """
    Ensambla reglas minimizando drawdown

    Args:
        df: DataFrame con features y target
        rules: lista de reglas validadas
        max_rules_per_ensemble: máximo de reglas a combinar

    Returns:
        dict con mejor ensemble
    """
    from itertools import combinations

    best_ensemble = None
    best_drawdown = float('inf')

    for n_rules in range(1, min(max_rules_per_ensemble + 1, len(rules) + 1)):
        for combo in combinations(rules, n_rules):
            signals = np.zeros(len(df), dtype=bool)

            for rule in combo:
                mask = (df[rule['feature']] >= rule['lower_bound']) & \
                    (df[rule['feature']] <= rule['upper_bound'])
                signals = signals | mask.values

            returns = df.loc[signals, 'Target']

            if len(returns) == 0:
                continue

            cumulative = (1 + returns).cumprod()
            running_max = cumulative.expanding().max()
            drawdown = (cumulative - running_max) / running_max
            max_dd = drawdown.min()

            if abs(max_dd) < best_drawdown:
                best_drawdown = abs(max_dd)
                best_ensemble = {
                    'rules': list(combo),
                    'max_drawdown': abs(max_dd),
                    'n_trades': len(returns),

```

```
        'mean_return': returns.mean(),  
        'total_return': returns.sum(),  
        'signals': signals  
    }  
  
    return best_ensemble
```

8.3. Métricas del Ensemble

El ensemble incluye:

- **rules** : lista de reglas combinadas
- **max_drawdown** : drawdown máximo alcanzado
- **n_trades** : número total de trades
- **mean_return** : retorno promedio por trade
- **total_return** : retorno total acumulado
- **sharpe_ratio** : ratio de Sharpe (opcional)

8.4. Alternativas de Ensamblado

Método	Descripción	Ventaja
OR Logic	Señal si cualquier regla se activa	Más trades, diversificación
AND Logic	Señal solo si todas las reglas se activan	Menos trades, mayor precisión
Voting	Señal si mayoría de reglas activas	Balance entre OR y AND
Weighted	Peso según performance individual	Prioriza mejores reglas

8.5. Interface Estándar

```
class RuleEnsemble:
    def create_ensemble(self, df: pd.DataFrame, rules: List[Dict],
                       method: str = 'or',
                       optimization_metric: str = 'drawdown') -> Dict:
        """
        Crea ensemble de reglas

        Args:
            df: DataFrame con features y target
            rules: lista de reglas validadas
            method: 'or', 'and', 'voting', 'weighted'
            optimization_metric: métrica a optimizar

        Returns:
            dict con configuración del ensemble
        """
        pass
```

9. Módulo 8: FILTERS (Pendiente)

9.1. Objetivo

Añadir filtros adicionales basados en contexto de mercado para mejorar la selectividad de las señales.

9.2. Filtros Propuestos

Filtro A: ADX (Average Directional Index)

Objetivo: Filtrar señales en mercados con poca tendencia.

Implementación propuesta:

```
def apply_adx_filter(df, ensemble_signals, adx_threshold=25):  
    """  
    Filtra señales por ADX  
  
    Args:  
        df: DataFrame con indicador ADX  
        ensemble_signals: señales del ensemble (bool array)  
        adx_threshold: umbral mínimo de ADX  
  
    Returns:  
        bool array con señales filtradas  
    """  
    adx_filter = df['ADX'] >= adx_threshold  
    filtered_signals = ensemble_signals & adx_filter.values  
  
    return filtered_signals
```

Configuración recomendada:

- ADX $\geq$ 25: Tendencia moderada o fuerte
- Solo operar cuando hay tendencia definida

Filtro B: ATR (Average True Range)

Objetivo: Ajustar position size según volatilidad.

Implementación propuesta:

```
def apply_atr_filter(df, ensemble_signals, atr_min=None, atr_max=None):
    """
    Filtra señales por rango de ATR

    Args:
        df: DataFrame con indicador ATR
        ensemble_signals: señales del ensemble
        atr_min: ATR mínimo (opcional)
        atr_max: ATR máximo (opcional)

    Returns:
        bool array con señales filtradas
    """
    atr_filter = pd.Series([True] * len(df), index=df.index)

    if atr_min is not None:
        atr_filter = atr_filter & (df['ATR'] >= atr_min)

    if atr_max is not None:
        atr_filter = atr_filter & (df['ATR'] <= atr_max)

    filtered_signals = ensemble_signals & atr_filter.values

    return filtered_signals
```

Casos de uso:

- **ATR mínimo:** Evitar operar en baja volatilidad
- **ATR máximo:** Evitar operar en volatilidad extrema

Filtro C: Time Filters (Futuro)

- Horarios de trading específicos
- Días de la semana
- Evitar eventos de alto impacto (NFP, FOMC)

9.3. Interface Estándar

```
class SignalFilter:
    def filter(self, df: pd.DataFrame, signals: np.ndarray,
              filter_config: dict) -> np.ndarray:
        """
        Aplica filtros a señales del ensemble

        Args:
            df: DataFrame con indicadores
            signals: señales del ensemble (bool array)
            filter_config: configuración de filtros

        Returns:
            bool array con señales filtradas
        """
        filtered = signals.copy()

        if 'adx' in filter_config:
            filtered = self.apply_adx_filter(df, filtered, **filter_config['adx'])

        if 'atr' in filter_config:
            filtered = self.apply_atr_filter(df, filtered, **filter_config['atr'])

        return filtered
```

10. Módulo 9: PARSER (Pendiente)

10.1. Objetivo

Traducir las reglas validadas a código ejecutable en diferentes plataformas.

10.2. Targets de Exportación

Target A: MQL5 (MetaTrader 5)

Objetivo: Generar Expert Advisor (EA) automático.

Estructura del EA:

```

//+-----+
//| Expert Advisor generado automáticamente por PyRE |
//+-----+

input double LotSize = 0.01;
input int MagicNumber = 12345;

// Parámetros de las reglas
input double Rule1_Lower = 0.3456;
input double Rule1_Upper = 0.5678;

int OnInit() {
    return(INIT_SUCCEEDED);
}

void OnTick() {
    // Calcular indicadores
    double indicator1 = iRSI(_Symbol, PERIOD_CURRENT, 14, PRICE_CLOSE, 0);

    // Evaluar reglas con lógica OR
    bool signal = false;

    if(indicator1 >= Rule1_Lower && indicator1 <= Rule1_Upper) {
        signal = true;
    }

    // Aplicar filtros
    double adx = iADX(_Symbol, PERIOD_CURRENT, 14, PRICE_CLOSE, 0);
    if(adx < 25) {
        signal = false;
    }

    if(signal) {
        OpenTrade();
    }
}

void OpenTrade() {
    // Lógica de apertura de posición
}

```

Implementación propuesta:

```
def export_to_mql5(rules, ensemble_config, filters_config, output_file):
    """
    Genera código MQL5 para EA

    Args:
        rules: lista de reglas del ensemble
        ensemble_config: configuración del ensemble
        filters_config: configuración de filtros
        output_file: ruta del archivo .mq5

    Returns:
        str con código MQL5 generado
    """
    mql5_code = []

    mql5_code.append("//+-----+")
    mql5_code.append("//| Expert Advisor - PyRE Generated |")
    mql5_code.append("//+-----+")
    mql5_code.append("")

    mql5_code.append("input double LotSize = 0.01;")
    mql5_code.append("input int MagicNumber = 12345;")
    mql5_code.append("")

    for i, rule in enumerate(rules, 1):
        mql5_code.append(f"input double Rule{i}_Lower = {rule['lower_bound']:.6f};")
        mql5_code.append(f"input double Rule{i}_Upper = {rule['upper_bound']:.6f};")
    mql5_code.append("")

    with open(output_file, 'w') as f:
        f.write('\n'.join(mql5_code))

    return '\n'.join(mql5_code)
```

Target B: Python Trading Bot**Objetivo:** Ejecutar directamente desde Python.**Implementación propuesta:**

```

def export_to_python_bot(rules, ensemble_config, filters_config):
    """
    Genera clase Python para trading bot

    Args:
        rules: lista de reglas del ensemble
        ensemble_config: configuración del ensemble
        filters_config: configuración de filtros

    Returns:
        str con código Python generado
    """
    py_code = []

    py_code.append("import pandas as pd")
    py_code.append("import numpy as np")
    py_code.append("from typing import List, Dict")
    py_code.append("")
    py_code.append("class TradingBot:")
    py_code.append("    def __init__(self):")

    py_code.append("        self.rules = []")
    for rule in rules:
        py_code.append(f"        {rule},")
    py_code.append("    ]")
    py_code.append("")

    py_code.append("    def evaluate_signal(self, df: pd.DataFrame) -> bool:")
    py_code.append("        \"\"\"Evalúa si hay señal de trading\"\"\"")
    py_code.append("        signal = False")
    py_code.append("")

    for i, rule in enumerate(rules, 1):
        feature = rule['feature']
        lower = rule['lower_bound']
        upper = rule['upper_bound']

        py_code.append(f"        # Regla {i}")
        py_code.append(f"        if df['{feature}'].iloc[-1] >= {lower} and \"")
        py_code.append(f"            df['{feature}'].iloc[-1] <= {upper}:")
        py_code.append(f"            signal = True")
        py_code.append("")

```

```
if filters_config:
    py_code.append("        # Aplicar filtros")
    if 'adx' in filters_config:
        threshold = filters_config['adx']['threshold']
        py_code.append(f"        if df['ADX'].iloc[-1] < {threshold}:")
        py_code.append("            signal = False")

py_code.append("")
py_code.append("    return signal")

return '\n'.join(py_code)
```

10.3. Validación del Parser

Tests esenciales:

1. ☒ Código generado compila sin errores
2. ☒ Indicadores se calculan correctamente
3. ☒ Lógica de reglas es equivalente al backtest
4. ☒ Filtros se aplican correctamente

10.4. Interface Estándar

```
class CodeParser:
    def parse(self, rules: List[Dict], ensemble_config: dict,
              filters_config: dict, target_platform: str) -> str:
        """
        Genera código para plataforma objetivo

        Args:
            rules: reglas del ensemble
            ensemble_config: configuración del ensemble
            filters_config: configuración de filtros
            target_platform: 'mql5', 'python', 'tradingview'

        Returns:
            str con código generado
        """
        if target_platform == 'mql5':
            return self.export_to_mql5(rules, ensemble_config, filters_config)
        elif target_platform == 'python':
            return self.export_to_python_bot(rules, ensemble_config, filters_config)
        else:
            raise ValueError(f"Platform {target_platform} not supported")
```

11. Flujos de Trabajo Típicos

11.1. Flujo A: Desarrollo Completo (CSV → MT5 EA)

Objetivo: Usuario con datos CSV que quiere desplegar en MT5.

Pipeline:

1. DATA: CSV Upload
↓
2. TARGET: Retorno Simple a 5 días
↓
3. INDICATORS: Usar indicadores del CSV
↓
4. FEATURE SELECTION: OneR con top 10
↓
5. EXTRACTION: OneR modificado, 5 cuantiles
↓
6. VALIDATION: Monkey Test + Synthetic Data
↓
7. ENSEMBLE: OR Logic minimizando DD
↓
8. FILTERS: ADX >= 25
↓
9. PARSER: Exportar a MQL5

Configuración sugerida:

```
pipeline_config = {
    'data': {
        'source': 'csv',
        'file_path': 'data/EURUSD_H1.csv'
    },
    'target': {
        'method': 'change_target',
        'days_ahead': 5
    },
    'indicators': {
        'source': 'csv'
    },
    'feature_selection': {
        'method': 'oner',
        'correlation_threshold': 0.85,
        'top_n': 10
    },
    'extraction': {
        'method': 'oner',
        'n_quantiles': 5,
        'min_samples': 30,
        'threshold_percentile': 60
    },
    'validation': {
        'monkey': {
            'n_permutations': 1000,
            'confidence_level': 0.95
        },
        'synthetic': {
            'n_synthetic': 1000,
            'method': 'bootstrap'
        }
    },
    'ensemble': {
        'method': 'or',
        'optimization_metric': 'drawdown',
        'max_rules': 5
    },
    'filters': {
        'adx': {
            'threshold': 25
        }
    }
}
```



```
},  
'parser': {  
    'target_platform': 'mql5',  
    'output_file': 'output/PyRE_EA.mq5'  
}  
}
```

11.2. Flujo B: Investigación Rápida (Python End-to-End)

Objetivo: Investigador que quiere iterar rápido usando Python puro.

Pipeline:

1. DATA: Quandl API
↓
2. TARGET: IBS Target a 3 días
↓
3. INDICATORS: pandas_ta
↓
4. FEATURE SELECTION: Mutual Information
↓
5. EXTRACTION: OneR modificado
↓
6. VALIDATION: Solo Monkey Test (rápido)
↓
7. ENSEMBLE: Weighted por Sharpe Ratio
↓
8. FILTERS: Ninguno (para análisis)
↓
9. PARSER: Python Bot

Configuración sugerida:

```
research_config = {
    'data': {
        'source': 'quandl',
        'symbol': 'CHRIS/CME_ES1',
        'start_date': '2020-01-01',
        'end_date': '2024-01-01',
        'api_key': 'YOUR_API_KEY'
    },
    'target': {
        'method': 'ibs_target',
        'days_ahead': 3
    },
    'indicators': {
        'source': 'pandas_ta',
        'indicators': ['rsi', 'macd', 'bbands', 'atr']
    },
    'feature_selection': {
        'method': 'mutual_info',
        'top_n': 15
    },
    'extraction': {
        'method': 'oner',
        'n_quantiles': 5
    },
    'validation': {
        'monkey': {
            'n_permutations': 500
        }
    },
    'ensemble': {
        'method': 'weighted',
        'optimization_metric': 'sharpe'
    },
    'filters': None,
    'parser': {
        'target_platform': 'python'
    }
}
```

11.3. Flujo C: MT5 Live Trading

Objetivo: Trader que quiere datos directos de MT5 y ejecutar allí.

Pipeline:

```
1. DATA: MT5 API
  ↓
2. TARGET: Retorno Simple a 1 día
  ↓
3. INDICATORS: Clonar indicadores MT5
  ↓
4. FEATURE SELECTION: OneR
  ↓
5. EXTRACTION: OneR modificado
  ↓
6. VALIDATION: Monkey Test + Synthetic
  ↓
7. ENSEMBLE: OR Logic
  ↓
8. FILTERS: ADX + ATR
  ↓
9. PARSER: MQL5 EA
```

Configuración sugerida:

```
mt5_live_config = {
    'data': {
        'source': 'mt5',
        'symbol': 'EURUSD',
        'timeframe': 'H1',
        'start_date': '2022-01-01',
        'end_date': '2024-01-01',
        'login': 12345,
        'password': 'your_password',
        'server': 'YourBroker-Live'
    },
    'target': {
        'method': 'change_target',
        'days_ahead': 1
    },
    'indicators': {
        'source': 'mt5_clone',
        'indicators': ['RSI_14', 'MACD', 'BB_20_2', 'ATR_14', 'ADX_14']
    },
    'feature_selection': {
        'method': 'oner',
        'top_n': 8
    },
    'extraction': {
        'method': 'oner',
        'n_quantiles': 5
    },
    'validation': {
        'monkey': {
            'n_permutations': 1000
        },
        'synthetic': {
            'n_synthetic': 1000
        }
    },
    'ensemble': {
        'method': 'or',
        'max_rules': 5
    },
    'filters': {
        'adx': {'threshold': 25},
        'atr': {'min': 0.0010, 'max': 0.0050}
    }
}
```

```
    },  
    'parser': {  
        'target_platform': 'mql5',  
        'output_file': 'MT5/Experts/PyRE_EURUSD_H1.mq5'  
    }  
}
```

12. Guía de Implementación

12.1. Estructura de Directorios Recomendada

```
PyRE_Framework/
|
|─ pyre/
|   |─ __init__.py
|   |─ data/
|   |   |─ __init__.py
|   |   |─ base.py
|   |   |─ csv_loader.py
|   |   |─ quandl_loader.py
|   |   └─ mt5_loader.py
|   |
|   |─ target/
|   |   |─ __init__.py
|   |   |─ base.py
|   |   |─ change_target.py
|   |   └─ ibs_target.py
|   |
|   |─ indicators/
|   |   |─ __init__.py
|   |   |─ base.py
|   |   |─ csv_indicators.py
|   |   |─ mt5_indicators.py
|   |   └─ pandas_ta_indicators.py
|   |
|   |─ feature_selection/
|   |   |─ __init__.py
|   |   |─ base.py
|   |   |─ oner_selector.py
|   |   |─ mi_selector.py
|   |   └─ correlation_selector.py
|   |
|   |─ extraction/
|   |   |─ __init__.py
|   |   |─ base.py
|   |   └─ oner_extractor.py
|   |
|   |─ validation/
|   |   |─ __init__.py
|   |   |─ base.py
|   |   |─ monkey_test.py
|   |   └─ synthetic_test.py
|   |
|   |
```

```
|   ├── ensemble/
|   |   ├── __init__.py
|   |   ├── base.py
|   |   └── or_ensemble.py
|   |
|   ├── filters/
|   |   ├── __init__.py
|   |   ├── base.py
|   |   ├── adx_filter.py
|   |   └── atr_filter.py
|   |
|   ├── parser/
|   |   ├── __init__.py
|   |   ├── base.py
|   |   ├── mql5_parser.py
|   |   └── python_parser.py
|   |
|   └── pipeline.py
|
└── app/
    ├── app.py
    └── utils.py
    |
└── configs/
    ├── full_development.yaml
    ├── quick_research.yaml
    └── mt5_live.yaml
    |
└── notebooks/
    ├── tutorial_01_basic.ipynb
    ├── tutorial_02_custom_pipeline.ipynb
    └── tutorial_03_extending.ipynb
    |
└── tests/
    ├── test_data_loaders.py
    ├── test_extractors.py
    └── test_validators.py
    |
└── examples/
    ├── example_csv_to_mql5.py
    ├── example_research.py
    └── example_custom_module.py
    |
```



```
|— docs/
|   |— PyRE_Framework_WhitePaper.md
|   |— api_reference.md
|   └─ contributing.md
|
|— requirements.txt
|— setup.py
└─ README.md
```

12.2. Clase Orquestadora Principal

```

# pyre/pipeline.py

from typing import Dict, Any, List
import pandas as pd
import yaml

class PyREPipeline:
    """
    Orquestador principal del framework PyRE
    """

    def __init__(self, config: Dict[str, Any]):
        if isinstance(config, str):
            with open(config, 'r') as f:
                self.config = yaml.safe_load(f)
        else:
            self.config = config

        self._init_modules()

        self.df = None
        self.selected_features = None
        self.rules = None
        self.validated_rules = None
        self.ensemble = None
        self.final_code = None

    def _init_modules(self):
        from pyre.data import DataLoaderFactory
        from pyre.target import TargetCalculatorFactory

        self.data_loader = DataLoaderFactory.create(
            self.config['data']['source']
        )
        self.target_calculator = TargetCalculatorFactory.create(
            self.config['target']['method']
        )

    def run(self) -> Dict[str, Any]:
        """Ejecuta el pipeline completo"""
        print("🚀 Iniciando Pipeline PyRE...\n")

```

```
print("📊 [1/9] Cargando datos...")
self.df = self.data_loader.load(**self.config['data'])
print(f"    ✓ Cargadas {len(self.df)} filas\n")

print("🎯 [2/9] Calculando target...")
self.df = self.target_calculator.calculate(self.df, **self.config['target'])
print(f"    ✓ Target calculado\n")

print("✅ Pipeline completado exitosamente!\n")

return {
    'df': self.df,
    'selected_features': self.selected_features,
    'rules': self.rules,
    'validated_rules': self.validated_rules,
    'ensemble': self.ensemble,
    'code': self.final_code
}
```

12.3. Ejemplo de Uso

```
# example_usage.py

from pyre import PyREPipeline

# Opción 1: Desde archivo de configuración
pipeline = PyREPipeline('configs/full_development.yaml')
results = pipeline.run()

# Opción 2: Configuración programática
config = {
    'data': {
        'source': 'csv',
        'file_path': 'data/EURUSD.csv'
    },
    'target': {
        'method': 'change_target',
        'days_ahead': 3
    },
}

pipeline = PyREPipeline(config)
results = pipeline.run()

print(f"Reglas validadas: {len(results['validated_rules'])}")
print(f"Ensemble: {results['ensemble']}")
```

12.4. Extensión: Añadir Nuevo Módulo

Ejemplo: Nuevo método de Feature Selection

```
# pyre/feature_selection/lasso_selector.py

from pyre.feature_selection.base import FeatureSelector
from sklearn.linear_model import LassoCV
import pandas as pd
from typing import List

class LassoFeatureSelector(FeatureSelector):
    """Feature selection usando Lasso Regression"""

    def select_top_features(self, df: pd.DataFrame, target_col: str,
                           top_n: int = 10) -> List[str]:
        X = df.drop(columns=[target_col])
        y = df[target_col]

        lasso = LassoCV(cv=5, random_state=42)
        lasso.fit(X, y)

        coef_abs = pd.Series(abs(lasso.coef_), index=X.columns)
        top_features = coef_abs.nlargest(top_n).index.tolist()

        return top_features
```

Registro en Factory:

```
# pyre/feature_selection/__init__.py

from pyre.feature_selection.one_selector import OneRSelector
from pyre.feature_selection.mi_selector import MutualInfoSelector
from pyre.feature_selection.lasso_selector import LassoFeatureSelector

class FeatureSelectorFactory:
    _selectors = {
        'oner': OneRSelector,
        'mutual_info': MutualInfoSelector,
        'lasso': LassoFeatureSelector
    }

    @classmethod
    def create(cls, method: str):
        if method not in cls._selectors:
            raise ValueError(f"Method {method} not supported")
        return cls._selectors[method]()
```

13. Roadmap y Extensiones Futuras

13.1. Prioridad Alta (Próximos 3 meses)

1. Completar Módulo FILTERS

-  Implementar filtro ADX
-  Implementar filtro ATR
-  Implementar filtros temporales
-  Implementar filtro de eventos

2. Completar Módulo PARSER

-  Parser MQL5 funcional
-  Parser Python Bot
-  Parser TradingView
-  Test suite para validar código

3. Módulo TARGET Avanzado

- 🕒 Retorno ajustado por costes
- 🕒 Target multinivel
- 🕒 Target basado en volatilidad

13.2. Prioridad Media (3-6 meses)

4. Walk-Forward Optimization

- 🕒 Implementar walk-forward analysis
- 🕒 Re-optimización periódica
- 🕒 Detección de degradación

5. Position Sizing Avanzado

- 🕒 Kelly Criterion
- 🕒 Risk Parity
- 🕒 Volatility-based sizing

6. Multi-Asset Support

- 🕒 Portfolio cross-asset
- 🕒 Correlación entre activos
- 🕒 Rotación de activos

13.3. Prioridad Baja (6-12 meses)

7. Machine Learning Integration

- 🕒 Ensemble con ML models
- 🕒 Feature engineering automático
- 🕒 Hyperparameter optimization

8. Cloud Deployment

- 🕒 Dockerización
- 🕒 API REST
- 🕒 Dashboard web

9. Community Extensions

- 🕒 Marketplace de módulos
- 🕒 Plugin system
- 🕒 Más brokers

13.4. Research Extensions

10. Nuevos Métodos de Extraction

- 🕒 Decision Trees
- 🕒 Association Rules
- 🕒 Rough Sets

11. Validación Avanzada

- 🕒 Cross-validation temporal
- 🕒 CPCV
- 🕒 Deflated Sharpe Ratio

14. Conclusiones

14.1. Ventajas de la Arquitectura Modular

1. **Flexibilidad:** Cada alumno personaliza su pipeline
2. **Extensibilidad:** Fácil añadir implementaciones
3. **Mantenibilidad:** Código organizado y testeable
4. **Reusabilidad:** Módulos independientes
5. **Educación:** Estructura clara

14.2. Principios de Diseño Clave

- ✅ **Interface-first**
- ✅ **Composition over inheritance**
- ✅ **Configuration-driven**

-  **Testability**
-  **Documentation**

14.3. Recomendaciones para Alumnos

Para Principiantes:

1. Empezar con CSV
2. Usar configs predefinidas
3. Entender cada módulo
4. Cambios incrementales

Para Avanzados:

1. Experimentar con combinaciones
2. Implementar módulos custom
3. Optimizar parámetros
4. Contribuir implementaciones

14.4. Próximos Pasos

1. **Migración:** Refactorizar código actual
2. **Testing:** Test suite completo
3. **Documentación:** API reference
4. **Tutoriales:** Notebooks interactivos
5. **Community:** GitHub + guidelines

Apéndices

Apéndice A: Glosario

- **OneR:** Algoritmo simple de ML
- **IBS:** Internal Bar Strength
- **Monkey Test:** Validación por permutaciones
- **Data Sintética:** Datos artificiales

- **Ensemble:** Combinación de reglas
- **Drawdown:** Caída desde máximo
- **EA:** Expert Advisor de MT5

Apéndice B: Referencias

- pandas: <https://pandas.pydata.org/>
- pandas_ta: <https://github.com/twopirllc/pandas-ta>
- scikit-learn: <https://scikit-learn.org/>
- MetaTrader5: <https://pypi.org/project/MetaTrader5/>
- plotly: <https://plotly.com/python/>

Apéndice C: FAQ

Q: ¿Puedo usar solo algunos módulos?

A: Sí, cada módulo es independiente.

Q: ¿Cómo añado mi método de extracción?

A: Implementa RuleExtractor y regístralo.

Q: ¿Soporta múltiples timeframes?

A: Actualmente uno a la vez.

Q: ¿Puedo usar indicadores custom?

A: Sí, añádelos como columnas.

Versión: 1.0

Fecha: Noviembre 2025

Autor: Jaume - Quantdemy

Licencia: [Por definir]

Documento completo para implementar PyRE Framework